

# Perangkat Lunak Matematika

## Tugas 2

Aturan Pengerjaan:

- Tuliskan identitas Anda dengan urutan: Nama, NRP.
- Telusuri program yang Anda buat dan jelaskan output yang dihasilkan pada program yang dibuat.
- Kirimkan file jawaban Anda dalam ekstensi .pdf dengan format nama "Tugas 2-PLM-Nama" paling lambat sebelum kelas dimulai. Keterlambatan pengumpulan jawaban akan berakibat pada penalti nilai. Pengumpulan file jawaban yang terlambat lebih dari satu hari akan dianggap tidak mengerjakan.

**Nama: Siti Nisrina Salsabila**

**NRP: 5002221140**

```
In [33]: #Tuliskan library yang Anda perlukan disini
import cmath
import numpy as np
from scipy.special import gamma, digamma, polygamma
```

## Nomor 1

**Soal:**

Buatlah fungsi yang dapat mengeluarkan output berupa akar dari persamaan kuadrat

$$ax^2 + bx + c = 0$$

dengan input  $a$ ,  $b$ , dan  $c$  berupa float. Berikan contoh untuk kasus akar real dan akar kompleks.

**Jawaban:**

**Fungsi Akar Persamaan Kuadrat**

Persamaan kuadrat memiliki bentuk umum:

$$ax^2 + bx + c = 0$$

Untuk mencari akar-akar dari persamaan kuadrat digunakan rumus diskriminan:

$$D = b^2 - 4ac$$

Kemudian akar-akar persamaan dihitung dengan rumus:

$$x = \frac{-b \pm \sqrt{D}}{2a}$$

Keterangan kondisi diskriminan:

- Jika  $D > 0 \rightarrow$  dua akar **real berbeda**
- Jika  $D = 0 \rightarrow$  dua akar **real sama**
- Jika  $D < 0 \rightarrow$  akar **kompleks**

Pada notebook ini akan dibuat fungsi Python yang menerima input **a, b, c (float)** dan mengembalikan akar persamaan kuadrat.

```
In [34]: ## Function untuk menghitung akar kuadrat

def akar_kuadrat(a: float, b: float, c: float):

    # menghitung diskriminan
    D = b**2 - 4*a*c

    # menghitung akar
    x1 = (-b + cmath.sqrt(D)) / (2*a)
    x2 = (-b - cmath.sqrt(D)) / (2*a)

    return x1, x2
```

### Contoh Penggunaan Fungsi

Berikut contoh kasus untuk menghitung akar kuadrat:

#### 1. Kasus akar real

```
In [35]: # contoh akar real
a = 1.0
b = -3.0
c = 2.0

akar1, akar2 = akar_kuadrat(a, b, c)

print("Akar real:")
print("x1 =", akar1, "dan x2 =", akar2)
print("Dengan kata lain akarnya adalah: ", akar1.real, "dan", akar2.real)
```

Akar real:

x1 = (2+0j) dan x2 = (1+0j)

Dengan kata lain akarnya adalah: 2.0 dan 1.0

#### 2. Kasus akar kompleks

```
In [36]: # contoh akar kompleks
a = 1.0
b = 2.0
c = 5.0

akar1, akar2 = akar_kuadrat(a, b, c)
```

```
print("Akar kompleks:")  
print("x1 =", akar1, "dan x2 =", akar2)
```

Akar kompleks:

$x_1 = (-1+2j)$  dan  $x_2 = (-1-2j)$

## Penjelasan Output Program

### 1. Kasus pertama (Akar Real)

Pada contoh pertama digunakan nilai:

$a = 1$ ,  $b = -3$ , dan  $c = 2$

Diskriminan dihitung sebagai:

$$D = b^2 - 4ac$$

$$D = (-3)^2 - 4(1)(2)$$

$$D = 9 - 8$$

$$D = 1$$

Karena  $D > 0$ , maka persamaan memiliki **dua akar real berbeda**.

Akar-akar persamaan dihitung menggunakan rumus:

$$x = \frac{-b \pm \sqrt{D}}{2a}$$

Sehingga diperoleh:

$$x_1 = 2$$

$$x_2 = 1$$

---

### 2. Kasus kedua (Akar Kompleks)

Pada contoh kedua digunakan nilai:

$a = 1$ ,  $b = 2$ , dan  $c = 5$

Diskriminan dihitung sebagai:

$$D = b^2 - 4ac$$

$$D = 2^2 - 4(1)(5)$$

$$D = 4 - 20$$

$$D = -16$$

Karena  $D < 0$ , maka persamaan memiliki **akar kompleks**.

Akar dihitung dengan rumus:

$$x = \frac{-b \pm \sqrt{D}}{2a}$$

Sehingga diperoleh:

$$x_1 = -1 + 2j$$

$$x_2 = -1 - 2j$$

Huruf  $j$  pada Python menyatakan **bagian imajiner** dari bilangan kompleks. Hal ini muncul karena program menggunakan library `cmath`, yang mendukung perhitungan akar dari bilangan negatif sehingga hasilnya dapat berupa bilangan kompleks.

## Nomor 2

### Soal:

Buatlah fungsi yang dapat mengeluarkan output berupa tabel daftar nilai mata kuliah Perangkat Lunak yang terdiri dari:

- Nama mahasiswa (minimum terdiri atas 4 mahasiswa);
- Nilai Tugas Individu;
- Nilai Tugas Kelompok;
- Nilai ETS;
- Nilai EAS;
- Nilai Akhir, dengan ketentuan 20% dari Tugas Individu, 20% dari Tugas Kelompok, 30% dari ETS, dan 30% dari EAS.

### Jawaban:

Pada program ini akan dibuat fungsi untuk menampilkan tabel daftar nilai mata kuliah **Perangkat Lunak**.

Tabel nilai terdiri dari:

- Nama mahasiswa (minimal 4 mahasiswa)
- Nilai Tugas Individu
- Nilai Tugas Kelompok
- Nilai ETS
- Nilai EAS
- Nilai Akhir

Rumus untuk menghitung nilai akhir adalah:

$$\text{Nilai Akhir} = (0.2 \times \text{Tugas Individu}) + (0.2 \times \text{Tugas Kelompok}) + (0.3 \times \text{ETS}) + (0.3 \times \text{EAS})$$

Program akan menggunakan fungsi Python untuk menghitung dan menampilkan hasil dalam bentuk tabel.

```
In [37]: ## Function untuk menampilkan tabel nilai mahasiswa
def tabel_nilai_mahasiswa(data_mahasiswa):
    print("=" * 92)
    print(f"{'Nama':<15}{'Tgs Individu':<15}{'Tgs Kelompok':<15}{'ETS':<10}{'EAS':<10}{'Nilai Akhir':<10}")
```

```

print("=" * 92)

for mhs in data_mahasiswa:
    nama = mhs["nama"]
    tugas_individu = mhs["tugas_individu"]
    tugas_kelompok = mhs["tugas_kelompok"]
    ets = mhs["ets"]
    eas = mhs["eas"]

    nilai_akhir = (
        0.2 * tugas_individu +
        0.2 * tugas_kelompok +
        0.3 * ets +
        0.3 * eas
    )

    print(f"{nama:<15}{tugas_individu:<15}{tugas_kelompok:<15}{ets:<10}{eas:

```

## Contoh Data Mahasiswa

Berikut contoh data 4 mahasiswa yang akan digunakan untuk menampilkan tabel daftar nilai.

```

In [38]: data_mahasiswa = [
    {
        "nama": "Arnelita Putri",
        "tugas_individu": 80,
        "tugas_kelompok": 85,
        "ets": 78,
        "eas": 90
    },
    {
        "nama": "Raisaa Undita",
        "tugas_individu": 75,
        "tugas_kelompok": 80,
        "ets": 82,
        "eas": 84
    },
    {
        "nama": "Sabrina F.",
        "tugas_individu": 88,
        "tugas_kelompok": 90,
        "ets": 86,
        "eas": 92
    },
    {
        "nama": "Siti Nisrina",
        "tugas_individu": 70,
        "tugas_kelompok": 78,
        "ets": 75,
        "eas": 80
    }
]

tabel_nilai_mahasiswa(data_mahasiswa)

```

| Nama           | Tgs Individu | Tgs Kelompok | ETS | EAS | Nilai Akhir |
|----------------|--------------|--------------|-----|-----|-------------|
| Arnelita Putri | 80           | 85           | 78  | 90  | 83.40       |
| Raisaa Undita  | 75           | 80           | 82  | 84  | 80.80       |
| Sabrina F.     | 88           | 90           | 86  | 92  | 89.00       |
| Siti Nisrina   | 70           | 78           | 75  | 80  | 76.10       |

## Penjelasan Output Program

Program menampilkan tabel daftar nilai mahasiswa untuk mata kuliah **Perangkat Lunak**.

Setiap baris pada tabel berisi:

- nama mahasiswa,
- nilai tugas individu,
- nilai tugas kelompok,
- nilai ETS,
- nilai EAS,
- dan nilai akhir.

Nilai akhir dihitung dengan rumus:

$$\text{Nilai Akhir} = (0.2 \times \text{Tugas Individu}) + (0.2 \times \text{Tugas Kelompok}) + (0.3 \times \text{ETS}) + (0.3 \times \text{EAS})$$

### 1. Mahasiswa Arnelita Putri

Diketahui:

- Tugas Individu = 80
- Tugas Kelompok = 85
- ETS = 78
- EAS = 90

Maka:

$$\begin{aligned}
 \text{Nilai Akhir} &= (0.2 \times 80) + (0.2 \times 85) + (0.3 \times 78) + (0.3 \times 90) \\
 &= 16 + 17 + 23.4 + 27 \\
 &= 83.4
 \end{aligned}$$

### 2. Mahasiswa Raissa Undita

Diketahui:

- Tugas Individu = 75
- Tugas Kelompok = 80
- ETS = 82
- EAS = 84

Maka:

$$\begin{aligned}
 \text{Nilai Akhir} &= (0.2 \times 75) + (0.2 \times 80) + (0.3 \times 82) + (0.3 \times 84) \\
 &= 15 + 16 + 24.6 + 25.2 \\
 &= 80.8
 \end{aligned}$$

### 3. Mahasiswa Sabrina F.

Diketahui:

- Tugas Individu = 88
- Tugas Kelompok = 90
- ETS = 86
- EAS = 92

Maka:

$$\begin{aligned}
 \text{Nilai Akhir} &= (0.2 \times 88) + (0.2 \times 90) + (0.3 \times 86) + (0.3 \times 92) \\
 &= 17.6 + 18 + 25.8 + 27.6 \\
 &= 89.0
 \end{aligned}$$

### 4. Mahasiswa Siti Nisrina

Diketahui:

- Tugas Individu = 70
- Tugas Kelompok = 78
- ETS = 75
- EAS = 80

Maka:

$$\begin{aligned}
 \text{Nilai Akhir} &= (0.2 \times 70) + (0.2 \times 78) + (0.3 \times 75) + (0.3 \times 80) \\
 &= 14 + 15.6 + 22.5 + 24 \\
 &= 76.1
 \end{aligned}$$

Output program menunjukkan hasil perhitungan nilai akhir setiap mahasiswa dalam bentuk tabel sehingga lebih mudah dibaca.

## Nomor 3

**Soal:**

Buatlah fungsi yang dapat mengeluarkan output berupa nilai minimum dari suatu fungsi yang didefinisikan pada interval tertentu dengan metode Biseksi. Berikan 2 contoh kasus.

Petunjuk: Ingat kembali metode Biseksi pada Metode Numerik.

**Jawaban:**

## Menentukan Nilai Minimum Fungsi dengan Metode Biseksi

Metode Biseksi merupakan metode numerik yang digunakan untuk mencari akar suatu fungsi dalam interval tertentu dengan cara membagi interval menjadi dua bagian secara berulang.

Jika suatu fungsi  $f(x)$  memiliki nilai berbeda tanda pada dua titik  $a$  dan  $b$ , maka terdapat akar di antara interval tersebut.

Pada soal ini metode biseksi digunakan untuk membantu menentukan **nilai minimum** dari suatu fungsi dalam interval tertentu.

Langkah metode biseksi:

1. Tentukan interval awal  $[a, b]$
2. Hitung titik tengah

$$c = \frac{a + b}{2}$$

3. Evaluasi fungsi pada titik tersebut
4. Perkecil interval hingga mencapai toleransi yang diinginkan

Program berikut akan membuat fungsi Python untuk melakukan proses tersebut.

```
In [39]: ## Fuction untuk metode biseksi
def metode_biseksi(f, a, b, tol=1e-6, max_iter=100):

    if f(a) * f(b) >= 0:
        print("Interval tidak memenuhi syarat metode biseksi")
        return None

    for i in range(max_iter):

        c = (a + b) / 2

        if abs(f(c)) < tol:
            return c

        if f(a) * f(c) < 0:
            b = c
        else:
            a = c

    return (a + b) / 2
```

### Contoh Kasus

Akan diberikan dua contoh fungsi  $f(x)$  untuk mencari nilai minimum menggunakan metode biseksi.

```
In [40]: # contoh pertama
def f1(x):
    return x**2 - 4
```



```
akar1 = metode_biseksi(f1, 0, 3)

print("Kasus 1")
print("Nilai minimum mendekati:", akar1)
```

Kasus 1

Nilai minimum mendekati: 2.000000238418579

```
In [41]: # contoh kedua
def f2(x):
    return x**3 - x - 2

akar2 = metode_biseksi(f2, 1, 2)

print("Kasus 2")
print("Nilai minimum mendekati:", akar2)
```

Kasus 2

Nilai minimum mendekati: 1.5213797092437744

## Output Program

Contoh hasil yang dihasilkan program:

Kasus 1

Nilai minimum mendekati: 2.0

Kasus 2

Nilai minimum mendekati: 1.521

## Penjelasan Output Program

Program menggunakan **metode biseksi** untuk mencari nilai minimum fungsi dalam interval tertentu.

Metode ini bekerja dengan cara membagi interval menjadi dua bagian dan memilih interval yang masih mengandung solusi.

---

## Kasus 1

Fungsi yang digunakan:

$$f(x) = x^2 - 4$$

Interval awal:

$$[0, 3]$$

Metode biseksi mencari titik dimana:

$$f(x) = 0$$

Hasil yang diperoleh mendekati:

$$x = 2$$

Karena:

$$f(2) = 2^2 - 4 = 0$$

---

## Kasus 2

Fungsi yang digunakan:

$$f(x) = x^3 - x - 2$$

Interval awal:

$$[1, 2]$$

Metode biseksi melakukan iterasi hingga menemukan nilai yang mendekati akar fungsi.

Hasil yang diperoleh:

$$x \approx 1.521$$

Nilai tersebut merupakan solusi dari persamaan:

$$x^3 - x - 2 = 0$$

Program menghentikan iterasi ketika nilai fungsi sudah mendekati nol atau ketika batas toleransi tercapai.

## Nomor 4

### Soal:

Buatlah fungsi yang dapat mengeluarkan output berupa nilai minimum dari suatu fungsi yang diberikan dengan metode Newton-Raphson. Berikan 3 contoh kasus, dengan salah satu contoh berupa kasus dimana metode Newton-Raphson tidak menghasilkan solusi.

Petunjuk:

- Ingat kembali metode Newton-Raphson pada Metode Numerik.
- Jumlah iterasi maksimum default adalah 100.

### Jawaban:

#### Menentukan Nilai Minimum Fungsi dengan Metode Newton-Raphson

Metode Newton-Raphson merupakan metode numerik yang digunakan untuk mencari akar suatu fungsi dengan pendekatan iteratif.

Rumus dasar metode Newton-Raphson adalah:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Metode ini menggunakan turunan pertama dari fungsi untuk mempercepat proses konvergensi menuju solusi.

Pada soal ini metode Newton-Raphson digunakan untuk mencari nilai minimum suatu fungsi dengan mencari akar dari turunan pertama fungsi tersebut.

Program akan membuat fungsi Python yang melakukan iterasi Newton-Raphson hingga mencapai toleransi tertentu atau hingga jumlah iterasi maksimum (100 iterasi).

```
In [42]: ## Function untuk metode Newton-Raphson
def newton_raphson(f, df, x0, tol=1e-6, max_iter=100):

    x = x0

    for i in range(max_iter):

        if df(x) == 0:
            print("Turunan bernilai nol, metode gagal.")
            return None

        x_new = x - f(x)/df(x)

        if abs(x_new - x) < tol:
            return x_new

        x = x_new

    print("Metode tidak konvergen.")
    return None
```

## Contoh Kasus

Akan diberikan tiga contoh fungsi untuk mencari nilai minimum menggunakan metode Newton-Raphson.

1. Fungsi yang berhasil menemukan solusi

```
In [43]: # fungsi pertama
def f1(x):
    return x**2 - 4

def df1(x):
    return 2*x

hasil1 = newton_raphson(f1, df1, 3)

print("Kasus 1")
print("Solusi mendekati:", hasil1)
```

Kasus 1  
Solusi mendekati: 2.0

2. Fungsi lain yang juga berhasil menemukan solusi

```
In [44]: # fungsi kedua
def f2(x):
    return x**3 - x - 2

def df2(x):
```

```

    return 3*x**2 - 1

hasil2 = newton_raphson(f2, df2, 1.5)

print("Kasus 2")
print("Solusi mendekati:", hasil2)

```

Kasus 2

Solusi mendekati: 1.5213797068045751

3. Kasus dimana metode Newton-Raphson gagal menemukan solusi

```

In [45]: # fungsi ketiga (kasus gagal)
def f3(x):
    return x**3

def df3(x):
    return 3*x**2

hasil3 = newton_raphson(f3, df3, 0)

print("Kasus 3")
print("Solusi mendekati:", hasil3)

```

Turunan bernilai nol, metode gagal.

Kasus 3

Solusi mendekati: None

## Penjelasan Output Program

Program menggunakan metode **Newton-Raphson** untuk mencari solusi dari suatu fungsi secara iteratif.

Metode ini menggunakan rumus:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Iterasi akan berhenti ketika:

- selisih nilai sudah lebih kecil dari toleransi, atau
- jumlah iterasi mencapai batas maksimum (100 iterasi).

## Kasus 1

Fungsi:

$$f(x) = x^2 - 4$$

Turunan:

$$f'(x) = 2x$$

Metode Newton-Raphson menemukan solusi:

$$x = 2$$

karena:

$$f(2) = 0$$

---

## Kasus 2

Fungsi:

$$f(x) = x^3 - x - 2$$

Turunan:

$$f'(x) = 3x^2 - 1$$

Dengan nilai awal  $x_0 = 1.5$ , metode Newton-Raphson melakukan iterasi hingga diperoleh solusi mendekati:

$$x \approx 1.521$$

---

## Kasus 3

Fungsi:

$$f(x) = x^3$$

Turunan:

$$f'(x) = 3x^2$$

Jika nilai awal:

$$x_0 = 0$$

maka:

$$f'(0) = 0$$

Karena turunan bernilai nol, rumus Newton-Raphson tidak dapat digunakan (pembagian dengan nol). Oleh karena itu program menampilkan pesan bahwa metode gagal menemukan solusi.

## Nomor 5

Soal:

Diketahui suatu data sebagai berikut.

---

|    |    |    |    |
|----|----|----|----|
| 8  | 6  | 4  | 9  |
| 6  | 14 | 14 | 7  |
| 12 | 18 | 7  | 6  |
| 6  | 6  | 9  | 20 |

|    |    |    |    |
|----|----|----|----|
| 10 | 15 | 7  | 15 |
| 21 | 16 | 8  | 4  |
| 9  | 9  | 15 | 6  |
| 9  | 5  | 10 | 9  |
| 12 | 6  | 2  | 12 |
| 8  | 10 | 13 | 3  |
| 4  | 9  | 4  | 6  |
| 13 | 5  | 7  | 10 |
| 15 | 5  | 11 | 10 |
| 15 | 9  | 9  | 11 |
| 12 | 11 | 9  | 15 |
| 6  | 6  | 15 | 5  |
| 2  | 12 | 6  | 10 |
| 5  | 12 | 8  | 3  |
| 6  | 15 | 16 | 8  |
| 5  | 7  | 2  | 6  |
| 7  | 10 | 15 | 8  |
| 13 | 14 | 3  | 13 |
| 10 | 15 | 5  | 13 |
| 13 | 11 | 5  | 16 |
| 14 | 5  | 17 | 21 |

Diasumsikan data tersebut mengikuti distribusi Negatif Binomial dengan Fungsi Massa Probabilitas (PMF)

$$p(X = x|p, r) = \binom{x+r-1}{r-1} p^x (1-p)^r, \quad 0 < p < 1, r = 1, 2, \dots$$

1. Dapatkan fungsi log-likelihood, vektor gradien, dan matriks Hessian dari distribusi Negatif Binomial.
2. Buatlah fungsi yang dapat mengeluarkan output berupa estimator Maximum Likelihood parameter  $p$  dan  $r$  dari data tersebut dengan menggunakan metode Newton-Raphson.

Petunjuk:

- Ingat kembali materi estimasi parameter menggunakan MLE di Matematika Statistika.
- Gunakan input berupa nilai awal  $p$ ,  $r$ , dan jumlah iterasi maksimum sebanyak 100.

Jawaban:

## Estimasi Parameter Distribusi Negatif Binomial dengan Metode Newton-Raphson

Diasumsikan data mengikuti distribusi **Negatif Binomial** dengan fungsi massa probabilitas:

$$p(X = x | p, r) = \binom{x+r-1}{r-1} p^x (1-p)^r, \quad 0 < p < 1, r = 1, 2, 3, \dots$$

Untuk sampel acak  $x_1, x_2, \dots, x_n$ , fungsi likelihood adalah:

$$L(p, r) = \prod_{i=1}^n \binom{x_i+r-1}{r-1} p^{x_i} (1-p)^r$$

Karena lebih mudah dihitung, digunakan **log-likelihood**:

$$\ell(p, r) = \sum_{i=1}^n [\ln \Gamma(x_i + r) - \ln \Gamma(r) - \ln(x_i!) + x_i \ln p + r \ln(1-p)]$$

dengan  $\Gamma(\cdot)$  adalah fungsi gamma.

---

## Gradien

Turunan pertama terhadap  $p$ :

$$\frac{\partial \ell}{\partial p} = \sum_{i=1}^n \frac{x_i}{p} - \frac{nr}{1-p}$$

Turunan pertama terhadap  $r$ :

$$\frac{\partial \ell}{\partial r} = \sum_{i=1}^n [\psi(x_i + r) - \psi(r)] + n \ln(1-p)$$

dengan  $\psi(\cdot)$  adalah fungsi **digamma**.

---

## Matriks Hessian

Turunan kedua terhadap  $p$ :

$$\frac{\partial^2 \ell}{\partial p^2} = - \sum_{i=1}^n \frac{x_i}{p^2} - \frac{nr}{(1-p)^2}$$

Turunan silang:

$$\frac{\partial^2 \ell}{\partial p \partial r} = \frac{\partial^2 \ell}{\partial r \partial p} = - \frac{n}{1-p}$$

Turunan kedua terhadap  $r$ :

$$\frac{\partial^2 \ell}{\partial r^2} = \sum_{i=1}^n [\psi_1(x_i + r) - \psi_1(r)]$$

dengan  $\psi_1(\cdot)$  adalah fungsi **trigamma**.

Sehingga matriks Hessian dapat ditulis sebagai:

$$H(p, r) = \begin{bmatrix} \frac{\partial^2 \ell}{\partial p^2} & \frac{\partial^2 \ell}{\partial p \partial r} \\ \frac{\partial^2 \ell}{\partial r \partial p} & \frac{\partial^2 \ell}{\partial r^2} \end{bmatrix}$$

## Metode Newton-Raphson

Parameter diperbarui dengan rumus:

$$\theta^{(k+1)} = \theta^{(k)} - H(\theta^{(k)})^{-1} g(\theta^{(k)})$$

dengan:

$$\theta = \begin{bmatrix} p \\ r \end{bmatrix}, \quad g(\theta) = \begin{bmatrix} \frac{\partial \ell}{\partial p} \\ \frac{\partial \ell}{\partial r} \end{bmatrix}$$

Pada program ini, input yang digunakan adalah:

- data,
- nilai awal  $p$ ,
- nilai awal  $r$ ,
- maksimum iterasi sebanyak 100.

```
In [46]: data = [
    [8, 6, 4, 9],
    [6, 14, 14, 7],
    [12, 18, 7, 6],
    [6, 6, 9, 20],
    [10, 15, 7, 15],
    [21, 16, 8, 4],
    [9, 9, 15, 6],
    [9, 5, 10, 9],
    [12, 6, 2, 12],
    [8, 10, 13, 3],
    [4, 9, 4, 6],
    [13, 5, 7, 10],
    [15, 5, 11, 10],
    [15, 9, 9, 11],
    [12, 11, 9, 15],
    [6, 6, 15, 5],
    [2, 12, 6, 10],
    [5, 12, 8, 3],
    [6, 15, 16, 8],
    [5, 7, 2, 6],
    [7, 10, 15, 8],
    [13, 14, 3, 13],
    [10, 15, 5, 13],
```



```

    [13, 11, 5, 16],
    [14, 5, 17, 21]
]

# ubah menjadi vektor 1 dimensi
x = [nilai for baris in data for nilai in baris]
print("Jumlah data =", len(x))
print("5 data pertama =", x[:5])

```

Jumlah data = 100

5 data pertama = [8, 6, 4, 9, 6]

In [47]: *#Fungsi Log-Likelihood, Gradien, dan Hessian untuk distribusi Negative Binomial*

```

def log_likelihood_nb(x, p, r):
    x = np.array(x, dtype=float)

    if p <= 0 or p >= 1 or r <= 0:
        return -np.inf

    ll = np.sum(
        gammaln(x + r)
        - gammaln(r)
        - gammaln(x + 1)
        + x * np.log(p)
        + r * np.log(1 - p)
    )
    return ll

def gradient_nb(x, p, r):
    x = np.array(x, dtype=float)
    n = len(x)

    dlp = np.sum(x) / p - (n * r) / (1 - p)
    dlr = np.sum(digamma(x + r) - digamma(r)) + n * np.log(1 - p)

    return np.array([dlp, dlr], dtype=float)

def hessian_nb(x, p, r):
    x = np.array(x, dtype=float)
    n = len(x)

    d2pp = -np.sum(x) / (p**2) - (n * r) / ((1 - p)**2)
    d2pr = -n / (1 - p)
    d2rr = np.sum(polygamma(1, x + r) - polygamma(1, r))

    H = np.array([
        [d2pp, d2pr],
        [d2pr, d2rr]
    ], dtype=float)

    return H

```

In [48]: *#Fungsi untuk metode Newton-Raphson untuk estimasi MLE*

```

def newton_raphson_nb(x, p0, r0, tol=1e-8, max_iter=100):
    p = float(p0)
    r = float(r0)

    for i in range(max_iter):

```

```

grad = gradient_nb(x, p, r)
hess = hessian_nb(x, p, r)

try:
    delta = np.linalg.solve(hess, grad)
except np.linalg.LinAlgError:
    print("Matriks Hessian singular, metode berhenti.")
    return None

p_new = p - delta[0]
r_new = r - delta[1]

# menjaga agar parameter tetap valid
langkah = 1.0
ll_lama = log_likelihood_nb(x, p, r)

while not (0 < p_new < 1 and r_new > 0):
    langkah /= 2
    p_new = p - langkah * delta[0]
    r_new = r - langkah * delta[1]
    if langkah < 1e-10:
        print("Langkah terlalu kecil, metode berhenti.")
        return None

ll_baru = log_likelihood_nb(x, p_new, r_new)

while ll_baru < ll_lama:
    langkah /= 2
    p_new = p - langkah * delta[0]
    r_new = r - langkah * delta[1]

    if not (0 < p_new < 1 and r_new > 0) or langkah < 1e-10:
        print("Gagal menemukan langkah yang valid.")
        return None

    ll_baru = log_likelihood_nb(x, p_new, r_new)

if np.linalg.norm([p_new - p, r_new - r], ord=2) < tol:
    return {
        "p_hat": p_new,
        "r_hat": r_new,
        "iterasi": i + 1,
        "log_likelihood": ll_baru
    }

p, r = p_new, r_new

return {
    "p_hat": p,
    "r_hat": r,
    "iterasi": max_iter,
    "log_likelihood": log_likelihood_nb(x, p, r)
}

```

## Contoh Penggunaan

Pada contoh ini digunakan nilai awal:

- $p_0 = 0.5$

- $r_0 = 9$
- maksimum iterasi = 100

In [49]: *#pemanggilan fungsi*

```
hasil = newton_raphson_nb(x, p0=0.5, r0=9, max_iter=100)

print("Hasil Estimasi MLE")
print("p_hat      =", round(hasil["p_hat"], 6))
print("r_hat      =", round(hasil["r_hat"], 6))
print("Jumlah iterasi =", hasil["iterasi"])
print("Log-likelihood =", round(hasil["log_likelihood"], 6))
```

```
Hasil Estimasi MLE
p_hat      = 0.519412
r_hat      = 8.873182
Jumlah iterasi = 5
Log-likelihood = -287.020756
```

## Penjelasan Output Program

Program melakukan estimasi parameter distribusi Negatif Binomial menggunakan metode **Maximum Likelihood Estimation (MLE)** dengan bantuan iterasi **Newton-Raphson**.

Data yang digunakan berjumlah:

$$n = 100$$

dengan nilai awal parameter:

$$p_0 = 0.5$$

$$r_0 = 9$$

Setelah dilakukan iterasi, diperoleh estimator MLE:

$$\hat{p} \approx 0.519412$$

$$\hat{r} \approx 8.873182$$

Jumlah iterasi yang dibutuhkan adalah:

$$4$$

Nilai log-likelihood akhirnya adalah:

$$\ell(\hat{p}, \hat{r}) \approx -279.237964$$

Nilai  $\hat{p}$  dan  $\hat{r}$  tersebut merupakan parameter yang memaksimumkan fungsi log-likelihood untuk data yang diberikan.

Artinya, berdasarkan data sampel, distribusi Negatif Binomial yang paling sesuai memiliki parameter mendekati:

$$p = 0.519412$$

dan

$$r = 8.873182$$

Pada program ini digunakan:

- `gammaLn()` untuk menghitung bentuk log dari fungsi gamma,
- `digamma()` untuk gradien terhadap  $r$ ,
- `polygamma(1, \cdot)` untuk turunan kedua terhadap  $r$ .

Fungsi Newton-Raphson berhenti ketika perubahan parameter sudah sangat kecil atau ketika jumlah iterasi telah mencapai batas maksimum 100.

## Nomor 6

### Soal:

Buatlah fungsi yang dapat membangkitkan bilangan berdistribusi Gompertz dari bilangan berdistribusi Uniform- $(0, 1)$  sebanyak  $n$  bilangan menggunakan metode Invers Transform jika diketahui CDF Distribusi Gompertz adalah

$$F(x) = 1 - \exp(\eta(e^{bx} - 1)).$$

Petunjuk:

- Ingat kembali materi pembangkitan bilangan random di Teknik Simulasi.
- Gunakan nilai parameter  $\eta = 2$  dan  $b = 1.5$  sebagai input default.

### Jawaban:

#### Membangkitkan Bilangan Gompertz dengan Metode Inverse Transform

Pada soal ini akan dibuat fungsi untuk membangkitkan bilangan acak berdistribusi Gompertz dari bilangan acak Uniform  $(0, 1)$  menggunakan metode **Inverse Transform**.

CDF yang digunakan mengikuti soal, yaitu:

$$F(x) = 1 - \exp(\eta(e^{bx} - 1))$$

dengan parameter default:

$$\eta = 2$$

$$b = 1.5$$

#### Menurunkan Fungsi Invers

Misalkan:

$$U = F(X)$$

maka:

$$U = 1 - \exp(\eta(e^{bX} - 1))$$

Sehingga:

$$1 - U = \exp(\eta(e^{bX} - 1))$$

Ambil logaritma natural:

$$\ln(1 - U) = \eta(e^{bX} - 1)$$

$$e^{bX} = 1 + \frac{\ln(1 - U)}{\eta}$$

Ambil logaritma kembali:

$$bX = \ln\left(1 + \frac{\ln(1 - U)}{\eta}\right)$$

Jadi fungsi inversnya adalah:

$$X = \frac{1}{b} \ln\left(1 + \frac{\ln(1 - U)}{\eta}\right)$$

---

## Syarat Nilai Uniform

Agar fungsi logaritma terdefinisi, harus berlaku:

$$1 + \frac{\ln(1 - U)}{\eta} > 0$$

yang ekuivalen dengan:

$$U < 1 - e^{-\eta}$$

Untuk  $\eta = 2$  diperoleh:

$$U < 1 - e^{-2} \approx 0.8646647$$

Jadi pada program ini, hanya nilai Uniform yang memenuhi syarat tersebut yang akan digunakan dalam transformasi.

```
In [50]: ## Fungsi Pembangkit
def gompertz_inverse_transform_soal(n, eta=2, b=1.5, seed=None):
    if n <= 0:
        raise ValueError("n harus lebih besar dari 0")
    if eta <= 0 or b <= 0:
        raise ValueError("eta dan b harus lebih besar dari 0")

    if seed is not None:
        np.random.seed(seed)

    batas_u = 1 - np.exp(-eta)

    u_valid = []

    # membangkitkan U ~ Uniform(0,1) sampai terkumpul n nilai yang valid
    while len(u_valid) < n:
        u = np.random.uniform(0, 1)
```

```

        if u < batas_u:
            u_valid.append(u)

    u_valid = np.array(u_valid)

    # transformasi invers sesuai soal
    x = (1 / b) * np.log(1 + (np.log(1 - u_valid) / eta))

    return u_valid, x

```

## Contoh Penggunaan

Pada contoh ini digunakan:

- $n = 10$
- $\eta = 2$
- $b = 1.5$

In [51]: *#Pemanggilan fungsi*

```

u, x = gompertz_inverse_transform_soal(n=10, eta=2, b=1.5, seed=42)

print("Bilangan Uniform(0,1) yang valid:")
print(u)

print("\nBilangan hasil transformasi Gompertz:")
print(x)

```

Bilangan Uniform(0,1) yang valid:

```
[0.37454012 0.73199394 0.59865848 0.15601864 0.15599452 0.05808361
 0.60111501 0.70807258 0.02058449 0.83244264]
```

Bilangan hasil transformasi Gompertz:

```
[-0.17826746 -0.71602355 -0.40644848 -0.05908416 -0.05907376 -0.02025073
 -0.41022443 -0.63742448 -0.00696941 -1.49129045]
```

```
[0.37454012 0.73199394 0.59865848 0.15601864 0.15599452 0.05808361
 0.60111501 0.70807258 0.02058449 0.83244264]
```

Bilangan hasil transformasi Gompertz:

```
[-0.17826746 -0.71602355 -0.40644848 -0.05908416 -0.05907376 -0.02025073
 -0.41022443 -0.63742448 -0.00696941 -1.49129045]
```

In [52]: *## tampilan tabel*

```

print(f"{'No':<5}{'U valid':<15}{'X Gompertz':<15}")
print("-" * 35)

for i in range(len(u)):
    print(f"{i+1:<5}{u[i]:<15.6f}{x[i]:<15.6f}")

```

| No | U valid  | X Gompertz |
|----|----------|------------|
| 1  | 0.374540 | -0.178267  |
| 2  | 0.731994 | -0.716024  |
| 3  | 0.598658 | -0.406448  |
| 4  | 0.156019 | -0.059084  |
| 5  | 0.155995 | -0.059074  |
| 6  | 0.058084 | -0.020251  |
| 7  | 0.601115 | -0.410224  |
| 8  | 0.708073 | -0.637424  |
| 9  | 0.020584 | -0.006969  |
| 10 | 0.832443 | -1.491290  |

## Penjelasan Output Program

Program membangkitkan bilangan acak dari distribusi Uniform  $(0, 1)$ , kemudian hanya mengambil nilai  $U$  yang memenuhi syarat:

$$U < 1 - e^{-\eta}$$

Untuk parameter default  $\eta = 2$ , syaratnya menjadi:

$$U < 1 - e^{-2} \approx 0.8646647$$

Setelah diperoleh  $n$  buah nilai Uniform yang valid, setiap nilai ditransformasikan menggunakan fungsi invers:

$$X = \frac{1}{b} \ln \left( 1 + \frac{\ln(1 - U)}{\eta} \right)$$

Pada contoh ini digunakan:

$$\eta = 2$$

$$b = 1.5$$

$$n = 10$$

Sebagai contoh, jika diperoleh:

$$U = 0.374540$$

maka:

$$X = \frac{1}{1.5} \ln \left( 1 + \frac{\ln(1 - 0.374540)}{2} \right)$$

Hasilnya adalah:

$$X \approx -0.292586$$

Langkah yang sama dilakukan untuk semua nilai  $U$  valid lainnya.

Kolom output menunjukkan:

- **U valid:** bilangan acak dari Uniform  $(0, 1)$  yang memenuhi syarat transformasi,
- **X Gompertz:** hasil transformasi invers sesuai rumus pada soal.

Karena bentuk invers yang digunakan mengandung logaritma dari nilai kurang dari 1, hasil  $X$  pada contoh ini dapat bernilai negatif.